INSTALLATION

```
#pip install transformers datasets scikit-learn accelerate
#pip install --upgrade transformers
#!pip install -U transformers
```

LIBRARY LOAD

```
import pandas as pd
import numpy as np
import torch
from datasets import Dataset
from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments, EarlyStoppingCallback
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from google.colab import drive
```

DATASET LOAD

```
# Dataset 1 লোড
df1 = pd.read_excel("/content/clean_dataset_1_agnews.xlsx")
df1 = df1[['text', 'label']]

# Dataset 1 এর লেবেলগুলোকে সংখ্যায় (0, 1, 2...) রূপান্তর করা
labels_list1 = df1['label'].unique().tolist()
label2id1 = {label: i for i, label in enumerate(labels_list1)}
df1['label'] = df1['label'].map(label2id1) # টেক্সট লেবেলগুলো সংখ্যা হয়ে যাবে

hf_dataset1 = Dataset.from_pandas(df1)
dataset_split1 = hf_dataset1.train_test_split(test_size=0.2, seed=42)
train_dataset1 = dataset_split1['train']
eval_dataset1 = dataset_split1['test']
num_labels1 = len(labels_list1)


# Dataset 2 লোড
df2 = pd.read_excel("/content/clean_dataset_1_agnews.xlsx")
df2 = df2[['text', 'label']]

# Dataset 2 এর লেবেলগুলোকে সংখ্যায় (0, 1, 2...) রূপান্তর করা
labels_list2 = df2['label'].unique().tolist()
label2id2 = {label: i for i, label in enumerate(labels_list2)}
df2['label'] = df2['label'].map(label2id2) # টেক্সট লেবেলগুলো সংখ্যা হয়ে যাবে

hf_dataset2 = Dataset.from_pandas(df2)
```

```
dataset_split2 = hf_dataset2.train_test_split(test_size=0.2, seed=42)
train_dataset2 = dataset_split2['train']
eval_dataset2 = dataset_split2['test']
num_labels2 = len(labels_list2)

print(f"Dataset 1 - Train: {len(train_dataset1)}, Eval: {len(eval_dataset1)}")
print(f"Dataset 1 Labels Mapping: {label2id1}")

print(f"\nDataset 2 - Train: {len(train_dataset2)}, Eval: {len(eval_dataset2)}")
print(f"Dataset 2 Labels Mapping: {label2id2}")
```

TOKENIZATION

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

def tokenize_function(examples):
    return tokenizer(examples['text'], padding='max_length', truncation=True, max_length=128)

tokenized_train1 = train_dataset1.map(tokenize_function, batched=True)
tokenized_eval1 = eval_dataset1.map(tokenize_function, batched=True)

tokenized_train2 = train_dataset2.map(tokenize_function, batched=True)
tokenized_eval2 = eval_dataset2.map(tokenize_function, batched=True)
```

EVALUATION METRICES

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)

    precision, recall, f1, _ = precision_recall_fscore_support(labels, predictions, average='weighted', zero_division=0)
    acc = accuracy_score(labels, predictions)

    return {'Accuracy': acc, 'Precision': precision, 'Recall': recall, 'F1': f1}
```

BERT APPLY

```
import math
from transformers import Trainer, TrainingArguments, BertForSequenceClassification, EarlyStoppingCallback

# BERT এর জন্য বাকি ৬টি কম্বিনেশন
bert_combinations = [
    {'lr': 0.00002, 'bs': 16, 'wd': 0.01},
    {'lr': 0.00003, 'bs': 16, 'wd': 0.01},
    {'lr': 0.00002, 'bs': 32, 'wd': 0.01},
    {'lr': 0.00003, 'bs': 32, 'wd': 0.01},
```

```
    {'lr': 0.00002, 'bs': 16, 'wd': 0.1},
    {'lr': 0.00003, 'bs': 16, 'wd': 0.1},
    {'lr': 0.00002, 'bs': 32, 'wd': 0.1},
    {'lr': 0.00003, 'bs': 32, 'wd': 0.1}
]

new_results_list = []

for combo in bert_combinations:
    lr, bs, wd = combo['lr'], combo['bs'], combo['wd']

    print(f"\n{'='*55}")
    print(f"BERT: LR={lr}, Batch={bs}, WD={wd} (Steps Only)")
    print(f"{'='*55}\n")

    # --- Dataset 1 ---
    print(">>> Training on Dataset 1...")
    model1 = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=num_labels1)

    args1 = TrainingArguments(
        output_dir=f'./BERT_Models/D1_lr{lr}_bs{bs}_wd{wd}',
        max_steps=100, learning_rate=lr,
        per_device_train_batch_size=bs, per_device_eval_batch_size=bs,
        weight_decay=wd, warmup_steps=10,
        eval_strategy="steps", eval_steps=25,
        save_strategy="steps", save_steps=25,
        load_best_model_at_end=True, metric_for_best_model="F1"
    )

    trainer1 = Trainer(model=model1, args=args1, train_dataset=tokenized_train1, eval_dataset=tokenized_eval1, compute_metrics=compute_metrics, callbacks=[EarlyStoppingCallback(ea
    trainer1.train()
    eval_results1 = trainer1.evaluate()

    # --- Dataset 2 ---
    print("\n>>> Training on Dataset 2...")
    model2 = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=num_labels2)

    args2 = TrainingArguments(
        output_dir=f'./BERT_Models/D2_lr{lr}_bs{bs}_wd{wd}',
        max_steps=100, learning_rate=lr,
        per_device_train_batch_size=bs, per_device_eval_batch_size=bs,
        weight_decay=wd, warmup_steps=10,
        eval_strategy="steps", eval_steps=25,
        save_strategy="steps", save_steps=25,
        load_best_model_at_end=True, metric_for_best_model="F1"
    )

    trainer2 = Trainer(model=model2, args=args2, train_dataset=tokenized_train2, eval_dataset=tokenized_eval2, compute_metrics=compute_metrics, callbacks=[EarlyStoppingCallback(ea
    trainer2.train()
    eval_results2 = trainer2.evaluate()

    # --- Save Results ---
    new_results_list.append({
        'Model': 'BERT', 'Learning Rate': lr, 'Batch Size': bs, 'Weight Decay': wd,
        'Dataset 1 Acc': eval_results1['eval_Accuracy'], 'Dataset 1 Prec': eval_results1['eval_Precision'],
```

```
        'Dataset 1 Rec': eval_results1['eval_Recall'], 'Dataset 1 F1': eval_results1['eval_F1'],
        'Dataset 2 Acc': eval_results2['eval_Accuracy'], 'Dataset 2 Prec': eval_results2['eval_Precision'],
        'Dataset 2 Rec': eval_results2['eval_Recall'], 'Dataset 2 F1': eval_results2['eval_F1']
    })
```

EXPORT TO DF

```
# ============================================================
# CREATE FINAL RESULTS DATAFRAME
# ============================================================

results_df = pd.DataFrame(results_list)

print("\n============================================================")
print("FINAL COMBINED RESULTS TABLE")
print("============================================================\n")

display(results_df)

# ============================================================
# SAVE RESULTS TO EXCEL
# ============================================================

file_name = "bert_experiments_combined_results.xlsx"

results_df.to_excel(
    file_name,
    index=False,
    engine="openpyxl"
)

print(
    f"\nফলাফল সফলভাবে '{file_name}' ফাইলে সেভ করা হয়েছে।"
)

# ============================================================
# TOTAL EXPERIMENTS
# ============================================================

print(
    f"\nTotal Hyperparameter Combinations Completed: {len(results_df)}"
)
```

Start coding or generate with AI.